МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

ЗВІТ

з лабораторної роботи № 3
«Розроблення підсистеми ідентифікації та автентифікації»
з дисципліни «Безпека програмного забезпечення»

Варіант 14

Виконав:
здобувач групи ІП-34
Скачков І. О.

Перевірив:
Курченко О. А.

Київ — 2026

1 МЕТА ТА ЗАВДАННЯ РОБОТИ

Мета роботи — набути практичних навичок проєктування та реалізації підсистеми ідентифікації й автентифікації користувача: безпечного зберігання паролів, кількісного обґрунтування їхньої стійкості, захисту від автоматизованого підбору та застосування другого фактора.

Завдання роботи:

1) розрахувати мінімальну довжину пароля за формулою Андерсона з параметрами варіанта;
2) реалізувати зберігання та перевірку пароля за KDF варіанта із сіллю, перцем і порівнянням за сталий час;
3) порівняти захищене зберігання з вразливим (SHA-256 без солі);
4) реалізувати блокування облікового запису за схемою варіанта;
5) реалізувати другий фактор варіанта й перевірити його;
6) розробити власний набір автоматизованих тестів.

Параметри варіанта отримано командою `python -m common.variants 14 --lab 3`:

```
  Варіант 14

    LAB3
      kdf: bcrypt
      second_factor: одноразові коди відновлення
      lockout: квота спроб у ковзному вікні
      anderson: period_days=120, guesses_per_minute=12, max_probability=1e-06
```

Таблиця 1.1 – Індивідуальне завдання (варіант 14)

Параметр	Значення
Функція формування ключа (KDF)	bcrypt
Другий фактор	одноразові коди відновлення
Схема блокування	квота спроб у ковзному вікні
Трійка Андерсона	T = 120 діб, R = 12 спроб/хв, P = 10^{-6}

2 ТЕОРЕТИЧНЕ ОБҐРУНТУВАННЯ ПРИЙНЯТИХ РІШЕНЬ

2.1 Параметри bcrypt і застосування перцю

Для bcrypt рекомендовано коефіцієнт складності cost ≥ 12; кожне збільшення cost на одиницю подвоює час обчислення (2^cost раундів). Обрано cost = 12: на стенді одне обчислення триває близько 0,2 с — непомітно під час входу, але дорого для перебору. Сіль генерує bcrypt.gensalt — 16 байтів із криптографічного ГВЧ ОС, що відповідає вимозі практикуму.

bcrypt має два обмеження: обробляє лише перші 72 байти пароля й непридатний для байтів NUL. Тому на вхід bcrypt подається не пароль, а base64(HMAC-SHA256(перець, пароль)) — рівно 44 ASCII-символи. Це водночас застосовує перець (через функцію _peppered каркаса) і усуває обрізання довгих паролів: пароль із 73 символів і пароль, що відрізняється лише 73-м символом, дають різні хеші (тест test_long_password_not_truncated). Перець зберігається у змінній середовища AUTH_PEPPER і в базу не потрапляє; у тестах і демонстрації задано лише тестове значення з коментарем «ТЕСТОВИЙ КЛЮЧ».

2.2 Квота спроб у ковзному вікні

Обліковому запису дозволено не більше 5 невдалих спроб за будь-які 15 хвилин. На відміну від фіксованого блокування, вікно «ковзає»: кожна невдача виходить із нього рівно через 15 хв після своєї появи, тож обмеження діє рівномірно, без моменту «скидання», яким можна скористатися на межі інтервалу. Параметри 5 / 15 хв обрано як типові для веб-систем: легітимний користувач рідко помиляється п’ять разів поспіль, а ефективна швидкість перебору знижується до 1/3 спроби за хвилину — у 36 разів нижче за R варіанта.

2.3 Одноразові коди відновлення

Генерується 10 кодів по 10 символів алфавіту Crockford Base32 (32 символи без I, L, O, U, що плутаються з цифрами), тобто $10 \cdot \log_2 32 = 50$ біт ентропії на код. Коди показуються один раз, а зберігаються лише HMAC-SHA256(перець, обліковий запис ‖ код): злив бази не розкриває кодів, а прив’язка до облікового

запису не дає використати код іншого користувача. Перевірка порівнює кандидата з усіма невикористаними хешами за сталий час без раннього виходу; успішно використаний код видаляється.

3 ОПИС РОЗРОБЛЕНОГО РІШЕННЯ

3.1 Структура та програмні засоби

```
SoS/lab3/lab03_auth/
  auth/anderson.py   формула Андерсона (каркас курсу, без змін)
  auth/passwords.py  hash_password / verify_password (bcrypt)
  auth/lockout.py    SlidingWindowLockout, record_attempt, is_allowed
  auth/recovery.py   генерація й перевірка кодів відновлення
  auth/service.py    Authenticator: пароль → код, спільна квота
  demo.py            демонстрація всіх кроків, графік L(A)
  tests/test_variant.py   30 тестових функцій (33 тестів)
```

Використано Python 3.12, bcrypt 4.3.0 (реалізація pyca/bcrypt; у requirements.txt каркаса бібліотеки для bcrypt немає, тому її додано), pytest 8.3.3, matplotlib 3.9.2 для графіка. Алгоритми блокування та кодів відновлення написано самостійно, лише на стандартній бібліотеці (secrets, hmac, hashlib).

3.2 Структурна схема

Схему процесу входу наведено на рис. 3.1. Обидва кроки — пароль і код — проходять одну перевірку квоти й записують невдачі в один лічильник облікового запису.

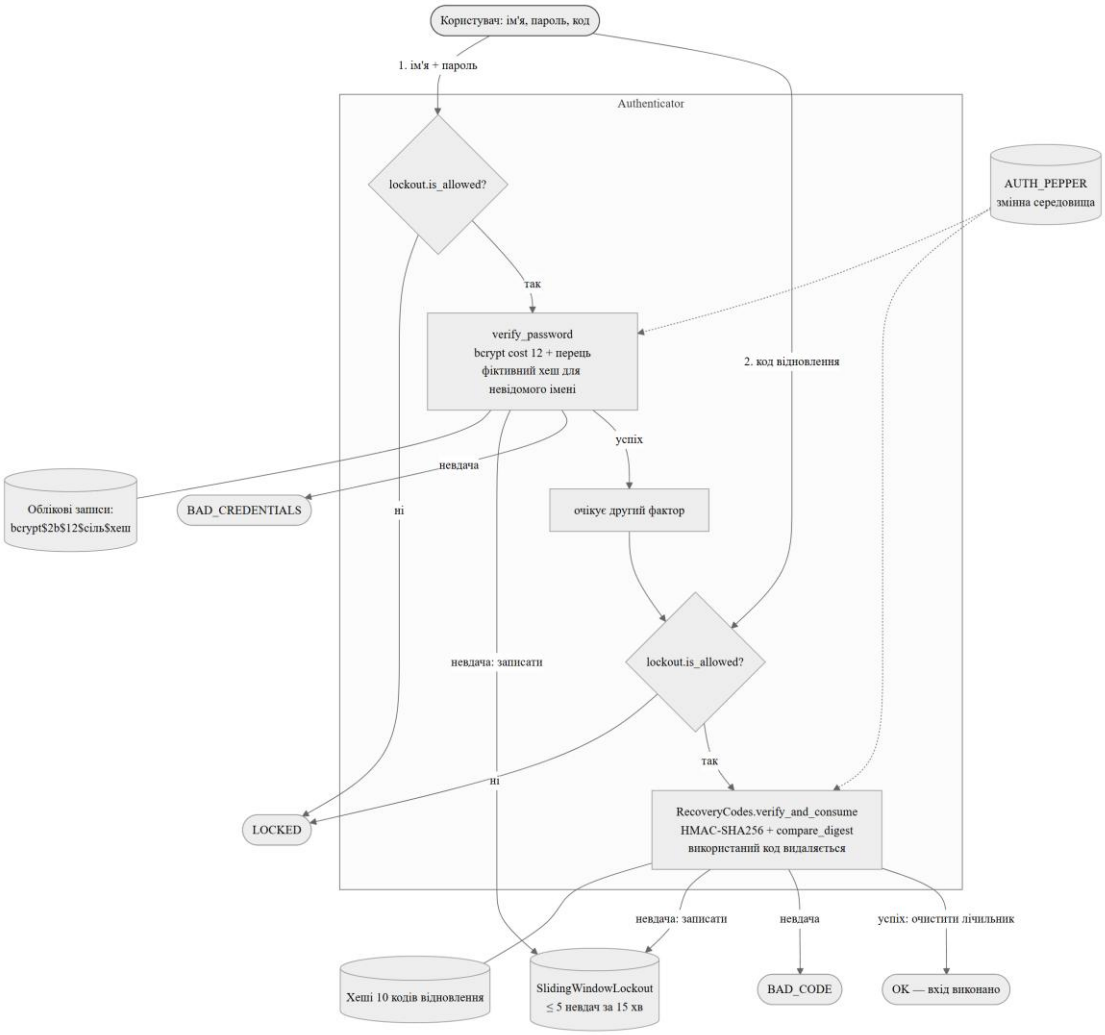


Рисунок 3.1 – Структурна схема підсистеми автентифікації

3.3 Зберігання та перевірка пароля

Формат збереженого рядка — `bcrypt$2b$<cost>$<сіль>$<хеш>`: назва схеми, версія алгоритму, коефіцієнт складності, 22 символи солі (16 байтів) і 31 символ хешу (23 байти) в алфавіті bcrypt. Приклад для пароля «correct horse»: `bcrypt$2b$12$.0tlgxCc3byqQ6/he8Ogdu$7GuEFVA.BLQOyEicAYlloc.j3xdZrFy.`

```
def hash_password(password: str, cost: int = COST) -> str:
    """БЕЗПЕЧНО: сіль (gensalt, 16 байтів з os.urandom) + перець + bcrypt."""
    salt = bcrypt.gensalt(rounds=cost)
    full = bcrypt.hashpw(_bcrypt_input(password), salt).decode()
    # full = "$2b$12$" + 22 символи солі + 31 символ хешу
    _, version, cost_s, rest = full.split("$")
    return f"{SCHEME}${version}${cost_s}${rest[:22]}${rest[22:]}"
```

```
def verify_password(stored: str, password: str) -> bool:
    """Перевірка пароля; хеші порівнюються лише за сталий час."""
    parsed = _parse(stored)
    if parsed is None:
        return False
```

```
    version, cost, salt, expected = parsed
    setting = f"${version}${cost:02d}${salt}".encode()
    computed = bcrypt.hashpw(_bcrypt_input(password), setting).decode()[-31:]
    return hmac.compare_digest(computed.encode(), expected.encode())
```

verify_password відновлює з рядка версію, cost і сіль, повторно обчислює bcrypt і порівнює результат функцією hmac.compare_digest. Некоректний рядок (інша схема, cost поза 4–31, неправильна довжина полів) не викликає винятку, а дає False. Підготовка входу bcrypt:

```
def _bcrypt_input(password: str) -> bytes:
    data = _peppered(password.encode("utf-8"))
    if not PEPPER:
        # без перцю — попереднє хешування, щоб bcrypt не обрізав довгий пароль
        data = hashlib.sha256(data).digest()
    return base64.b64encode(data)    # 44 байти, без NUL
```

3.4 Блокування

SlidingWindowLockout зберігає для кожного облікового запису чергу моментів невдалих спроб; перед кожним запитом із черги видаляються ті, що старші за вікно:

```
def _prune(self, account: str, now: float) -> collections.deque[float]:
    q = self._failures[account]
    while q and now - q[0] >= self.window:
        q.popleft()
    return q

def record_attempt(self, account: str, ok: bool, now: float) -> None:
    if ok:
        self._failures.pop(account, None)
    else:
        self._prune(account, now).append(now)

def is_allowed(self, account: str, now: float) -> bool:
    return len(self._prune(account, now)) < self.max_failures
```

Модульні функції record_attempt(account, ok, now) та is_allowed(account, now) з шаблону практикуму делегують екземпляру за замовчуванням. Спроби, відхилені через блокування, пароль не перевіряють і в лічильник не записують, тож порушник не може безкінечно продовжувати блокування; успішний вхід очищує лічильник.

3.5 Коди відновлення

```
def generate_codes(count: int = COUNT,
                   randbelow: Callable[[int], int] = secrets.randbelow,
                   ) -> list[str]:
    """Список одноразових кодів; randbelow — криптографічний ГВЧ."""
    codes = []
```

```
    while len(codes) < count:
        raw = "".join(ALPHABET[randbelow(len(ALPHABET))] for _ in range(CODE_LEN))
        code = f"{raw[:5]}-{raw[5:]}"
        if code not in codes:
            codes.append(code)
    return codes
```

```
def verify_and_consume(self, code: str) -> bool:
    candidate = code_hash(self.account, code).encode()
    match = -1
    for i, h in enumerate(self._hashes):          # без раннього виходу
        if hmac.compare_digest(h.encode(), candidate) and match < 0:
            match = i
    if match < 0:
        return False
    del self._hashes[match]                       # одноразовість
    return True
```

3.6 Процес входу

Authenticator.login_password перевіряє квоту, потім пароль; для неіснуючого імені перевірка виконується з фіктивним хешем, щоб час відповіді й повідомлення не розкривали існування облікового запису. Після правильного пароля лічильник не очищується — це відбувається лише після правильного коду, інакше, знаючи пароль, можна було б скидати квоту й необмежено перебирати коди.

```
def login_password(self, account: str, password: str, now: float) -> Result:
    if not self.lockout.is_allowed(account, now):
        return Result.LOCKED
    acc = self._accounts.get(account)
    stored = acc.password_hash if acc else self._dummy
    ok = verify_password(stored, password) and acc is not None
    if not ok:
        self.lockout.record_attempt(account, False, now)
        return Result.BAD_CREDENTIALS
    if needs_rehash(acc.password_hash):
        acc.password_hash = hash_password(password)
    self._pending.add(account)
    return Result.NEED_SECOND_FACTOR
```

4 РЕЗУЛЬТАТИ

4.1 Розрахунок за формулою Андерсона

Період дії в хвилинах $T = 120 \cdot 24 \cdot 60 = 172\,800$ хв; кількість спроб за період $R \cdot T = 12 \cdot 172\,800 = 2\,073\,600$. Нижня межа простору паролів за (4.1):

$$G \geq R \cdot T / P = 2\,073\,600 / 10^{-6} = 2{,}0736 \cdot 10^{12}.$$

Мінімальна довжина за (4.2) $L = \lceil \ln G / \ln A \rceil$, $\ln G = 28{,}360$; ентропія за (4.3) $H = L \cdot \log_2 A$ (табл. 4.1). Вивід demo.py, що виконує розрахунок модулем auth/anderson.py:

```
== 1. Формула Андерсона (R = 12 спроб/хв, T = 120 діб, P = 1e-6)
```

```
T = 120 · 24 · 60 = 172800 хв; R · T = 12 · 172800 = 2073600
G ≥ R·T / P = 2073600 / 1e-6 = 2.0736e+12
  A = 26: L = ⌈ln G / ln A⌉ = ⌈28.360 / 3.258⌉ = ⌈8.705⌉ = 9; H = 9 · log2 26 = 42.3 біт
  A = 62: L = ⌈ln G / ln A⌉ = ⌈28.360 / 4.127⌉ = ⌈6.872⌉ = 7; H = 7 · log2 62 = 41.7 біт
  A = 95: L = ⌈ln G / ln A⌉ = ⌈28.360 / 4.554⌉ = ⌈6.228⌉ = 7; H = 7 · log2 95 = 46.0 біт
З квотою 5 невдач / 15 хв: R_eff = 0.333 спроб/хв, G ≥ 5.7600e+10
  A = 26: L = ⌈7.605⌉ = 8; H = 37.6 біт
  A = 62: L = ⌈6.003⌉ = 7; H = 41.7 біт
  A = 95: L = ⌈5.441⌉ = 6; H = 39.4 біт
```

Таблиця 4.1 – Мінімальна довжина пароля та ентропія

A	ln A	ln G / ln A	L	H, біт	L з квотою	H з квотою, біт
26	3,258	8,705	9	42,3	8	37,6
62	4,127	6,872	7	41,7	7	41,7
95	4,554	6,228	7	46,0	6	39,4

Стовпці «з квотою» обчислено для ефективної швидкості R_eff = 5 / 15 = 1/3 спроби/хв, яку допускає схема блокування: G_eff ≥ 5,76 · 10¹⁰. Залежність L(A) для обох швидкостей наведено на рис. 4.1.

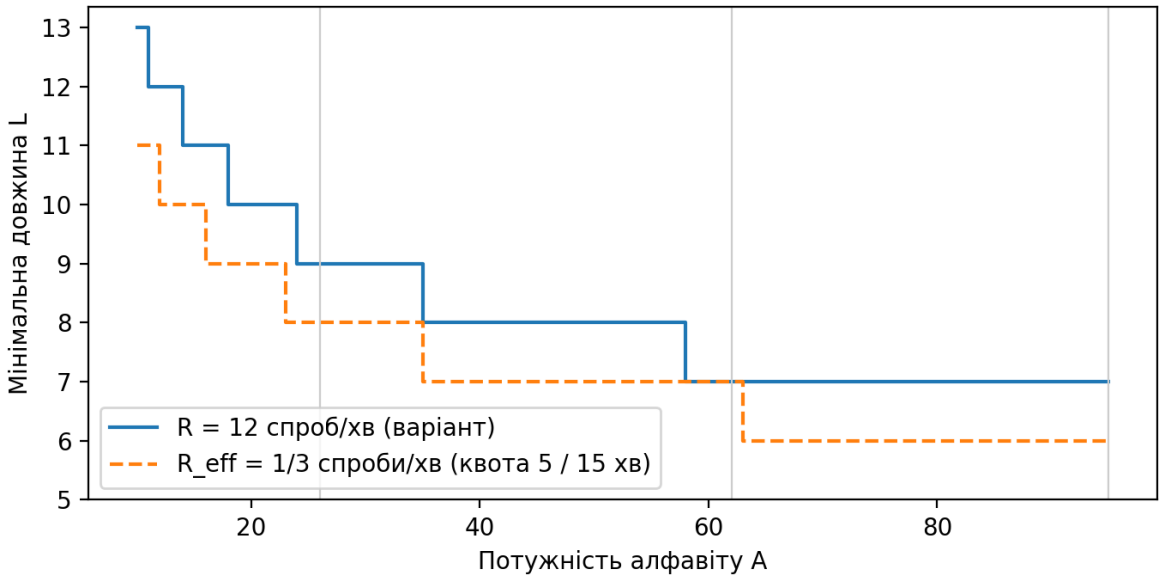


Рисунок 4.1 – Мінімальна довжина пароля залежно від потужності алфавіту

4.2 Захищене й вразливе зберігання

Два виклики hash_password("correct horse") дали різні рядки `bcrypt$2b$12$.0tlgxCc3byqQ6/he8Ogdu$7GuEFVA.BLQOyEicAYlloc.j3xdZrFy` і `bcrypt$2b$12$jyiFCE4HQ53sKuK5k0P6fO$gBOc2GMKLOCyurfXXhRlKCPXMd93Szu`; verify_password повертає True для правильного пароля й False для хибного. Вразливий hash_password_insecure("password123") двічі дає той самий рядок `ef92b778bafe771e89245b89ecbc08a4…` — це SHA-256 відомого слабкого пароля,

який є в будь-якій райдужній таблиці. Експеримент для двох облікових записів з однаковим паролем: зі спільною сіллю хеші bcrypt збігаються, з різними солями — ні (тест test_shared_salt_reveals_equal_passwords).

4.3 Час обчислення

Таблиця 4.2 – Час обчислення хешу пароля

Функція	Час одного обчислення	Відносно SHA-256
SHA-256 (hash_password_insecure)	0,59 мкс	1
bcrypt, cost 12 (hash_password)	198,0 мс (σ = 2,3 мс, n = 5)	≈ 337 315
bcrypt, cost 10	54 мс	—
bcrypt, cost 11	107 мс	—
bcrypt, cost 12	213 мс	—
bcrypt, cost 13	425 мс	—

Перебір 10^9 пароль-кандидатів на одному ядрі триває близько 10 хв для SHA-256 і близько 6,3 року для bcrypt з cost 12.

4.4 Сценарій блокування

П’ять невдалих спроб через кожні 60 с, далі перевірка is_allowed на межах вікна (табл. 4.3):

Таблиця 4.3 – Поведінка квоти 5 невдач / 15 хв на межах

t, с	Дозволено	Стан
0	так	запис невдачі №1
60	так	запис невдачі №2
120	так	запис невдачі №3
180	так	запис невдачі №4
240	так	запис невдачі №5
240.5	ні	невдач у вікні: 5; чекати 659.5 с
899.9	ні	невдач у вікні: 5; чекати 0.1 с
900.0	так	невдач у вікні: 4; чекати 0.0 с
959.9	так	невдач у вікні: 4; чекати 0.0 с
960.0	так	невдач у вікні: 3; чекати 0.0 с

Блокування настає одразу після п’ятої невдачі; о 899,9 с спроба ще заборонена, а рівно о 900 с перша невдача виходить із вікна й дозволяється одна спроба. Очікування retry_after = 659,5 с о 240,5 с збігається з розрахунком 0 + 900 − 240,5.

4.5 Коди відновлення та повний вхід

Згенеровано 10 синтетичних кодів: NNSQV-X32AV, Q5NW3-C3EHD, Q9S3R-PK0CB, E2SAX-20BRV, MR9E5-41GFB, AHDYV-YHVKC, CD57T-D1X71, ZRPWS-1KBA0, NZH6H-NQQ5P, TE131-9FJE7. Перше пред'явлення codes[0] прийнято (залишилось 9), повторне — відхилено (залишилось 9); код, введений малими літерами з пробілом замість дефіса, прийнято після нормалізації. Повний сценарій входу з табл. 4.4 показує спільну квоту: 2 хибні паролі та 3 хибні коди вичерпують 5 спроб, і навіть правильний пароль о 60 с відхиляється.

Таблиця 4.4 – Сценарій входу «пароль → код»

t, с	Крок	Введено	Результат
0	пароль	wrong	BAD_CREDENTIALS
10	пароль	wrong	BAD_CREDENTIALS
20	пароль	Tr0ub4dor&3	NEED_SECOND_FACTOR
30	код	00000-00000	BAD_CODE
40	код	11111-11111	BAD_CODE
50	код	22222-22222	BAD_CODE
60	пароль	Tr0ub4dor&3	LOCKED
900	пароль, код	Tr0ub4dor&3, codes[0]	NEED_SECOND_FACTOR, OK (кодів: 9)

4.6 Тестування

Файл tests/test_variant.py містить 30 тестових функцій (з урахуванням параметризації — 33 тести): формула Андерсона (3), зберігання паролів (13 функцій, зокрема зворотність, унікальність солі, відмова для хибного пароля, застосування перцю, сталий час, неприйняття пошкодженого рядка, спільна сіль), квота спроб (4), коди відновлення (6), повний вхід (4). Результат `PYTHONPATH=. pytest tests/ -q`: 33 passed; повний звіт — у додатку Б.

5 АНАЛІЗ РЕЗУЛЬТАТІВ

5.1 Залежність L від потужності алфавіту

$L = \lceil \ln G / \ln A \rceil$ — спадна ступінчаста функція, обернено пропорційна ln A (рис. 4.1). Через логарифм виграш від розширення алфавіту швидко зменшується: перехід від 26 до 62 символів скорочує довжину з 9 до 7, а від 62 до 95 — не змінює

її (6,87 і 6,23 округлюються до 7). Вимога насправді стосується ентропії: $H \geq \log_2 G \approx 40{,}9$ біт за будь-якого A, а алфавіт визначає лише, скількома символами її набрати; тому фактична H у табл. 4.1 (41,7–46,0 біт) трохи перевищує межу через округлення L вгору.

5.2 Вартість обчислення bcrypt

Одне обчислення hash_password триває 198,0 мс, SHA-256 — 0,59 мкс, тобто bcrypt повільніший приблизно в 337 315 разів. Для легітимного користувача це 0,2 с на вхід, для порушника — та сама вартість на кожен пароль-кандидат: перебір 10^9 кандидатів на ядрі зростає з 10 хв до 6,3 року. Табл. 4.2 підтверджує експоненційне зростання: кожне +1 до cost подвоює час, тож параметр можна підвищувати разом зі зростанням продуктивності обладнання (needs_rehash оновлює хеш під час наступного входу).

5.3 Сіль і перець

hash_password_insecure детермінована: однаковий пароль завжди дає однаковий SHA-256, тому злита база показує, у кого паролі збігаються, і дає змогу шукати хеші в заздалегідь обчислених райдужних таблицях. hash_password щоразу генерує нову сіль, тож однакові паролі дають різні рядки. Сіль захищає від попередніх обчислень і від масового перебору всієї бази одним проходом: кожен хеш доводиться атакувати окремо. Від цільового перебору одного хешу сіль не захищає, бо зберігається поряд із ним. Перець — спільний секрет поза базою: без нього порушник, що має лише вміст бази, не може перевірити жодного кандидата (тест test_other_pepper_fails). Тому їх зберігають у різних місцях: сіль мусить бути доступна разом із хешем, а перець — лише серверу застосунку.

5.4 Блокування та формула Андерсона

Квота спрацьовує після 5-ї невдалої спроби в межах 15 хвилин: 6-та спроба відхиляється без перевірки пароля. Для одного облікового запису ефективна швидкість падає з R = 12 до R_eff = 1/3 спроби/хв (у 36 разів), G — з $2{,}07 \cdot 10^{12}$ до $5{,}76 \cdot 10^{10}$, а потрібна довжина для A = 95 — з 7 до 6 символів, для A = 26 — з 9 до

8; для A = 62 довжина лишається 7, бо ln G_eff / ln 62 = 6,003 ледь перевищує 6. Недолік схеми — ризик відмови в обслуговуванні: порушник, що знає лише ім'я, надсилає 5 хибних паролів кожні 15 хвилин і постійно тримає легітимного користувача заблокованим; оскільки коди відновлення рахуються в ту саму квоту, заблокованим виявляється й резервний шлях. Квота також не стримує розпилення паролів (password spraying) по багатьох облікових записах. Пом'якшення — квота за парою «обліковий запис + IP», CAPTCHA після вичерпання квоти, сповіщення власника.

5.5 Порівняння за сталий час

Оператор == завершує порівняння на першому відмінному символі, тож час відповіді залежить від довжини спільного префікса. Вимірюючи затримку, порушник відновлює очікуване значення посимвольно: для коду відновлення з 10 символів Base32 це близько $10 \cdot 32 = 320$ спроб замість $32^{10} \approx 1{,}1 \cdot 10^{15}$. У реалізації порівнюються HMAC-хеші, а не самі коди, тому витік префікса хешу не дає прямо підібрати код, але hmac.compare_digest і цикл без раннього виходу прибирають і цей канал, і різницю в часі залежно від того, котрий за порядком код збігся. Тест test_constant_time_compare_used перевіряє, що verify_password викликає саме hmac.compare_digest.

6 ВИСНОВКИ

1) Для трійки варіанта (T = 120 діб, R = 12 спроб/хв, P = 10^{-6}) отримано $G \geq 2{,}0736 \cdot 10^{12}$, мінімальну довжину 9 / 7 / 7 символів для алфавітів 26 / 62 / 95 і ентропію 42,3 / 41,7 / 46,0 біт; контрольний приклад практикуму (L = 7, 46 біт) відтворено.
2) Реалізовано зберігання паролів за bcrypt (cost 12, 16-байтова сіль, перець через HMAC-SHA256 з base64-кодуванням для обходу межі 72 байти) з перевіркою hmac.compare_digest; одне обчислення триває 198,0 мс, що в ≈ 337 315 разів повільніше за SHA-256.

3) Квота 5 невдач у ковзному вікні 15 хв блокує 6-ту спробу й знову допускає спробу рівно через 900 с після першої невдачі; вона знижує ефективну швидкість перебору в 36 разів і потрібну довжину для A = 95 до 6 символів, але дає ризик відмови в обслуговуванні.
4) Реалізовано 10 одноразових кодів відновлення по 50 біт, що зберігаються лише як HMAC-хеші; кожен код приймається один раз, кількість дійсних кодів після використання зменшується на одиницю, а хибні паролі й коди вичерпують спільну квоту.
5) Власний набір із 33 тестів (30 функцій) проходить повністю.

7 ВІДОМОСТІ ПРО ВИКОРИСТАННЯ ГЕНЕРАТИВНИХ ІНСТРУМЕНТІВ

Під час виконання роботи як допоміжний засіб використано генеративну модель Claude (Anthropic): для підготовки чернетки коду модулів passwords.py, lockout.py, recovery.py, service.py, тестів і демонстраційного сценарію, а також оформлення звіту. Результати перевірено запуском тестів і демонстрації; рішення здобувач перевірив і може пояснити під час захисту.

ПЕРЕЛІК ПОСИЛАНЬ

1) Anderson R. Security Engineering: A Guide to Building Dependable Distributed Systems. – 3rd ed. – Indianapolis : Wiley, 2020. – Ch. 3 (Passwords).
2) NIST SP 800-63B. Digital Identity Guidelines: Authentication and Lifecycle Management. – Gaithersburg : NIST, 2017 (with updates 2020).
3) Provos N., Mazières D. A Future-Adaptable Password Scheme // Proceedings of the USENIX Annual Technical Conference. – 1999. – P. 81–91.
4) OWASP Password Storage Cheat Sheet [Електронний ресурс] / OWASP Foundation. – Режим доступу: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

5) Anderson J. P. Computer Security Technology Planning Study. – Bedford : ESD-TR-73-51, USAF Electronic Systems Division, 1972. – Vol. II.
6) Безпека програмного забезпечення. Частина 2. Лабораторний практикум / В. В. Соколовський. – Київ : КПІ ім. Ігоря Сікорського, 2026.

ДОДАТОК А

Репозиторій і артефакти роботи

Репозиторій: https://github.com/ip34is/7-sem (папка SoS/lab3)

Коміт, що відповідає зданій роботі:

`a47b0c112890294643205f6971360c056053f32c`

Стан роботи на цьому коміті: https://github.com/ip34is/7-sem/tree/a47b0c112890294643205f6971360c056053f32c/SoS/lab3

Артефакти: lab03_auth/auth/passwords.py, lockout.py, recovery.py, service.py, tests/test_variant.py, demo.py; у каталозі звіт_лр3 — scheme.mmd / scheme.png, anderson_plot.png, demo_output.txt, pytest_output.txt. Усі паролі, перець і коди в артефактах — синтетичні тестові значення.

Відтворення результатів (з папки SoS/lab3/lab03_auth):

```
pip install -r requirements.txt
PYTHONPATH=. pytest tests/ -q          # 33 passed
python demo.py --plot ../звіт_лр3/anderson_plot.png
```

ДОДАТОК Б

Звіт pytest для tests/test_variant.py

```
test_anderson_lecture_example PASSED
test_anderson_variant_14 PASSED
test_lockout_lowers_required_space PASSED
test_roundtrip PASSED
test_wrong_password_rejected PASSED
test_salt_is_unique PASSED
test_storage_format_and_cost PASSED
test_pepper_is_applied PASSED
test_other_pepper_fails PASSED
test_long_password_not_truncated PASSED
test_unicode_password PASSED
test_constant_time_compare_used PASSED
test_malformed_stored_value_rejected[] PASSED
test_malformed_stored_value_rejected[bcrypt$2b$12$short$x] PASSED
test_malformed_stored_value_rejected[sha256$2b$12$aaaaaaaaaaaaaaaaaaaaaa$bbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbb] PASSED
test_malformed_stored_value_rejected[bcrypt$2b$99$aaaaaaaaaaaaaaaaaaaaaa$bbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbb] PASSED
test_needs_rehash_for_weaker_cost PASSED
test_insecure_storage_reveals_equal_passwords PASSED
test_shared_salt_reveals_equal_passwords PASSED
test_lockout_triggers_exactly_at_limit PASSED
test_lockout_window_boundary PASSED
test_lockout_is_sliding_not_fixed PASSED
test_lockout_success_resets_and_accounts_are_independent PASSED
test_ten_codes_generated_with_expected_format PASSED
test_code_generation_is_deterministic_with_given_rng PASSED
test_each_code_accepted_once_and_count_decreases PASSED
test_wrong_and_foreign_codes_rejected PASSED
test_code_normalization PASSED
test_only_hashes_are_stored PASSED
test_full_login_with_password_and_code PASSED
test_code_without_password_step_refused PASSED
test_password_and_code_failures_share_quota PASSED
test_unknown_account_same_answer_and_counted PASSED
============================== 33 passed in 8.63s ==============================
```

ДОДАТОК В

Вивід demo.py

```
== 1. Формула Андерсона (R = 12 спроб/хв, T = 120 діб, P = 1e-6)
T = 120 · 24 · 60 = 172800 хв; R · T = 12 · 172800 = 2073600
G ≥ R·T / P = 2073600 / 1e-6 = 2.0736e+12
  A = 26: L = ⌈ln G / ln A⌉ = ⌈28.360 / 3.258⌉ = ⌈8.705⌉ = 9; H = 9 · log2 26 = 42.3 біт
  A = 62: L = ⌈ln G / ln A⌉ = ⌈28.360 / 4.127⌉ = ⌈6.872⌉ = 7; H = 7 · log2 62 = 41.7 біт
  A = 95: L = ⌈ln G / ln A⌉ = ⌈28.360 / 4.554⌉ = ⌈6.228⌉ = 7; H = 7 · log2 95 = 46.0 біт
З квотою 5 невдач / 15 хв: R_eff = 0.333 спроб/хв, G ≥ 5.7600e+10
  A = 26: L = ⌈7.605⌉ = 8; H = 37.6 біт
  A = 62: L = ⌈6.003⌉ = 7; H = 41.7 біт
  A = 95: L = ⌈5.441⌉ = 6; H = 39.4 біт


== 2. Захищене зберігання (bcrypt, cost 12, сіль, перець)
hash_password('correct horse') = bcrypt$2b$12$.0tlgxCc3byqQ6/he8Ogdu$7GuEFVA.BLQOyEicAYlloc.j3xdZrFy
ще раз                         = bcrypt$2b$12$jyiFCE4HQ53sKuK5k0P6fO$gBOc2GMKLOCyurfXXhRlKCPXMd93Szu
verify(правильний): True | verify(хибний): False | солі різні: True


== 3. Вразливе зберігання (SHA-256 без солі)
hash_password_insecure('password123') = ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
однакові паролі → однаковий хеш: True
hash_password дає різні рядки:    True
Експеримент: два облікові записи, однаковий пароль 'password123'
  спільна сіль, спільний перець → хеші однакові: True
  різні солі, спільний перець   → хеші однакові: False


== 4. Час обчислення (середнє)
hash_password (bcrypt, cost 12): 198.0 мс (n = 5, σ = 2.3 мс)
SHA-256:                          0.59 мкс (n = 200000)
bcrypt повільніший у ≈ 337 315 разів
Перебір 1e9 паролів на 1 ядрі: SHA-256 ≈ 10 хв, bcrypt ≈ 6.3 років
  cost 10: 54 мс
  cost 11: 107 мс
  cost 12: 213 мс
  cost 13: 425 мс


== 5. Квота спроб у ковзному вікні (5 невдач / 15 хв)
  t =    0 с: дозволено = True, невдача №1
```

```
  t =   60 с: дозволено = True, невдача №2
  t =  120 с: дозволено = True, невдача №3
  t =  180 с: дозволено = True, невдача №4
  t =  240 с: дозволено = True, невдача №5
  t =  240.5 с: дозволено = False, невдач у вікні = 5, чекати 659.5 с
  t =  899.9 с: дозволено = False, невдач у вікні = 5, чекати 0.1 с
  t =  900.0 с: дозволено = True, невдач у вікні = 4, чекати 0.0 с
  t =  959.9 с: дозволено = True, невдач у вікні = 4, чекати 0.0 с
  t =  960.0 с: дозволено = True, невдач у вікні = 3, чекати 0.0 с


== 6. Одноразові коди відновлення (синтетичні, лише для демонстрації)
Згенеровано: NNSQV-X32AV, Q5NW3-C3EHD, Q9S3R-PK0CB, E2SAX-20BRV, MR9E5-41GFB, AHDYV-YHVKC, CD57T-D1X71, ZRPWS-1KBA0, NZH6H-NQQ5P,
TE131-9FJE7
Збережено хешів: 10, приклад: 88338f82a0152d60f885545a586b4668…
Пред'явлення codes[0]: True → залишилось 9
Повторно codes[0]:    False → залишилось 9
Код з малими літерами і пробілом: True → залишилось 8


== 7. Повний вхід: пароль → код, спільна квота
  t =   0 с пароль wrong         → BAD_CREDENTIALS
  t =  10 с пароль wrong         → BAD_CREDENTIALS
  t =  20 с пароль Tr0ub4dor&3   → NEED_SECOND_FACTOR
  t =  30 с код    00000-00000   → BAD_CODE
  t =  40 с код    11111-11111   → BAD_CODE
  t =  50 с код    22222-22222   → BAD_CODE
  t =  60 с пароль Tr0ub4dor&3   → LOCKED
  t = 900 с пароль → NEED_SECOND_FACTOR, код → OK, залишилось кодів 9


Графік L(A) збережено: ../звіт_лр3/anderson_plot.png
```

ДОДАТОК Г

Лістинги модулів auth/passwords.py, lockout.py, recovery.py, service.py

```
# auth/passwords.py
"""Зберігання та перевірка паролів — ЛР3, варіант 14: bcrypt.

Формат збереженого рядка:  bcrypt$2b$<cost>$<сіль 22 симв.>$<хеш 31 симв.>
Сіль (16 байтів) і хеш записано в алфавіті bcrypt (./A-Za-z0-9), як їх видає бібліотека.

Вхід bcrypt — не сам пароль, а base64(HMAC-SHA256(перець, пароль)):
  - перець не потрапляє в базу, тож злита база без сервера непридатна для перебору;
  - 44 ASCII-байти без NUL обходять обмеження bcrypt (72 байти, обрізання довших паролів).
"""
from __future__ import annotations

import base64
import hashlib
import hmac
import os

import bcrypt

PEPPER = os.environ.get("AUTH_PEPPER", "").encode()

COST = 12                       # коефіцієнт складності: 2^12 раундів (рекомендація ≥ 12)
SCHEME, VERSION = "bcrypt", "2b"



# ВРАЗЛИВИЙ зразок (для порівняння у звіті) — НЕ використовувати:
def hash_password_insecure(password: str) -> str:
    # ВРАЗЛИВО: швидка хеш-функція без солі — вразлива до райдужних таблиць.
    return hashlib.sha256(password.encode()).hexdigest()



def hash_password(password: str, cost: int = COST) -> str:
```

```
    """БЕЗПЕЧНО: сіль (gensalt, 16 байтів з os.urandom) + перець + bcrypt."""
    salt = bcrypt.gensalt(rounds=cost)
    full = bcrypt.hashpw(_bcrypt_input(password), salt).decode()
    # full = "$2b$12$" + 22 символи солі + 31 символ хешу
    _, version, cost_s, rest = full.split("$")
    return f"{SCHEME}${version}${cost_s}${rest[:22]}${rest[22:]}"



def verify_password(stored: str, password: str) -> bool:
    """Перевірка пароля; хеші порівнюються лише за сталий час."""
    parsed = _parse(stored)
    if parsed is None:
        return False
    version, cost, salt, expected = parsed
    setting = f"${version}${cost:02d}${salt}".encode()
    computed = bcrypt.hashpw(_bcrypt_input(password), setting).decode()[-31:]
    return hmac.compare_digest(computed.encode(), expected.encode())



def needs_rehash(stored: str, cost: int = COST) -> bool:
    """True, якщо рядок створено зі слабшими параметрами, ніж чинні."""
    parsed = _parse(stored)
    return parsed is None or parsed[1] < cost



def _parse(stored: str) -> tuple[str, int, str, str] | None:
    parts = stored.split("$")
    if len(parts) != 5 or parts[0] != SCHEME or parts[1] != VERSION:
        return None
    _, version, cost_s, salt, digest = parts
    if not cost_s.isdigit() or not 4 <= int(cost_s) <= 31 or len(salt) != 22 or len(digest) != 31:
        return None
    return version, int(cost_s), salt, digest



def _bcrypt_input(password: str) -> bytes:
    data = _peppered(password.encode("utf-8"))
    if not PEPPER:
        # без перцю — попереднє хешування, щоб bcrypt не обрізав довгий пароль
        data = hashlib.sha256(data).digest()
```

```
    return base64.b64encode(data)    # 44 байти, без NUL



def _peppered(data: bytes) -> bytes:
    return hmac.new(PEPPER, data, hashlib.sha256).digest() if PEPPER else data
```

```
# auth/lockout.py
"""Блокування облікового запису — ЛР3, варіант 14: квота спроб у ковзному вікні.


Обліковому запису дозволено не більше `max_failures` невдалих спроб за будь-які
`window` секунд. Невдача в момент t враховується, поки now - t < window, тобто
рівно через window секунд вона «виходить» із вікна. Успішний вхід очищує лічильник.
Спроби, відхилені через блокування, пароль не перевіряють і в лічильник не потрапляють.
Стан — у пам'яті процесу.
"""
from __future__ import annotations


import collections


MAX_FAILURES = 5          # квота невдалих спроб
WINDOW = 15 * 60          # ковзне вікно, с



class SlidingWindowLockout:
    def __init__(self, max_failures: int = MAX_FAILURES, window: float = WINDOW) -> None:
        self.max_failures = max_failures
        self.window = window
        self._failures: dict[str, collections.deque[float]] = collections.defaultdict(collections.deque)


    def _prune(self, account: str, now: float) -> collections.deque[float]:
        q = self._failures[account]
        while q and now - q[0] >= self.window:
            q.popleft()
        return q


    def record_attempt(self, account: str, ok: bool, now: float) -> None:
        if ok:
            self._failures.pop(account, None)
        else:
            self._prune(account, now).append(now)
```

```
    def is_allowed(self, account: str, now: float) -> bool:
        return len(self._prune(account, now)) < self.max_failures

    def failures(self, account: str, now: float) -> int:
        return len(self._prune(account, now))

    def retry_after(self, account: str, now: float) -> float:
        """Через скільки секунд буде дозволена наступна спроба (0 — уже дозволена)."""
        q = self._prune(account, now)
        if len(q) < self.max_failures:
            return 0.0
        # має вийти з вікна стільки найстаріших невдач, щоб їх лишилося max_failures - 1
        return q[len(q) - self.max_failures] + self.window - now


_default = SlidingWindowLockout()


def record_attempt(account: str, ok: bool, now: float) -> None:
    _default.record_attempt(account, ok, now)


def is_allowed(account: str, now: float) -> bool:
    return _default.is_allowed(account, now)
# auth/recovery.py
"""Другий фактор — ЛР3, варіант 14: одноразові коди відновлення.

- 10 кодів по 10 символів алфавіту Crockford Base32 (без I, L, O, U): 10 · 5 = 50 біт кожен;
  показуються у вигляді XXXXX-XXXXX один раз, під час генерації.
- Зберігаються лише HMAC-SHA256(перець, обліковий запис ‖ код): злив бази не розкриває кодів,
  а прив'язка до облікового запису не дає використати чужий хеш.
- Перевірка порівнює кандидата з УСІМА невикористаними хешами за сталий час і не зупиняється
  на першому збігу; використаний код видаляється — повторно він не приймається.
"""
from __future__ import annotations

import hashlib
```

```
import hmac
import secrets
from collections.abc import Callable


from auth.passwords import PEPPER

ALPHABET = "0123456789ABCDEFGHJKMNPQRSTVWXYZ"   # Crockford Base32
CODE_LEN = 10
COUNT = 10
_NORMALIZE = str.maketrans({"O": "0", "I": "1", "L": "1", "-": None, " ": None})



def generate_codes(count: int = COUNT,
                   randbelow: Callable[[int], int] = secrets.randbelow,
                   ) -> list[str]:
    """Список одноразових кодів; randbelow — криптографічний ГВЧ."""
    codes = []
    while len(codes) < count:
        raw = "".join(ALPHABET[randbelow(len(ALPHABET))] for _ in range(CODE_LEN))
        code = f"{raw[:5]}-{raw[5:]}"
        if code not in codes:
            codes.append(code)
    return codes



def normalize(code: str) -> str:
    return code.upper().translate(_NORMALIZE)



def code_hash(account: str, code: str) -> str:
    msg = f"{account}\x00{normalize(code)}".encode()
    return hmac.new(PEPPER, msg, hashlib.sha256).hexdigest()



class RecoveryCodes:
    """Невикористані коди облікового запису (лише хеші)."""

    def __init__(self, account: str, codes: list[str]) -> None:
        self.account = account
```

```
        self._hashes: list[str] = [code_hash(account, c) for c in codes]

    @classmethod
    def issue(cls, account: str, count: int = COUNT) -> tuple["RecoveryCodes", list[str]]:
        """Нові коди: повертає сховище хешів і відкриті коди для показу користувачеві."""
        codes = generate_codes(count)
        return cls(account, codes), codes


    def remaining(self) -> int:
        return len(self._hashes)


    def verify_and_consume(self, code: str) -> bool:
        candidate = code_hash(self.account, code).encode()
        match = -1
        for i, h in enumerate(self._hashes):          # без раннього виходу
            if hmac.compare_digest(h.encode(), candidate) and match < 0:
                match = i
        if match < 0:
            return False
        del self._hashes[match]                       # одноразовість
        return True
```

```
# auth/service.py
"""Підсистема автентифікації: пароль (bcrypt) → код відновлення, з квотою спроб.

Обидва кроки рахуються в одну квоту облікового запису: інакше, знаючи пароль,
можна було б перебирати коди без обмеження. Для неіснуючого облікового запису
виконується перевірка з фіктивним хешем, щоб час відповіді не розкривав, чи існує запис,
і повідомлення про помилку однакові.
"""
from __future__ import annotations

import dataclasses
import enum

from auth.lockout import SlidingWindowLockout
from auth.passwords import hash_password, needs_rehash, verify_password
from auth.recovery import RecoveryCodes



class Result(enum.Enum):
```

```
    LOCKED = "забагато невдалих спроб, спробуйте пізніше"
    BAD_CREDENTIALS = "неправильне ім'я або пароль"
    NEED_SECOND_FACTOR = "пароль прийнято, введіть код відновлення"
    BAD_CODE = "код недійсний або вже використаний"
    NO_PENDING = "спершу введіть пароль"
    OK = "вхід виконано"



@dataclasses.dataclass
class Account:
    password_hash: str
    codes: RecoveryCodes



class Authenticator:
    def __init__(self, lockout: SlidingWindowLockout | None = None) -> None:
        self.lockout = lockout or SlidingWindowLockout()
        self._accounts: dict[str, Account] = {}
        self._pending: set[str] = set()          # пароль прийнято, чекаємо другий фактор
        self._dummy = hash_password("dummy-password-for-timing")


    def register(self, account: str, password: str) -> list[str]:
        """Створює обліковий запис; повертає коди відновлення (показати один раз)."""
        codes, plain = RecoveryCodes.issue(account)
        self._accounts[account] = Account(hash_password(password), codes)
        return plain


    def login_password(self, account: str, password: str, now: float) -> Result:
        if not self.lockout.is_allowed(account, now):
            return Result.LOCKED
        acc = self._accounts.get(account)
        stored = acc.password_hash if acc else self._dummy
        ok = verify_password(stored, password) and acc is not None
        if not ok:
            self.lockout.record_attempt(account, False, now)
            return Result.BAD_CREDENTIALS
        if needs_rehash(acc.password_hash):
            acc.password_hash = hash_password(password)
        self._pending.add(account)
        return Result.NEED_SECOND_FACTOR
```

```
def login_code(self, account: str, code: str, now: float) -> Result:
    if not self.lockout.is_allowed(account, now):
        return Result.LOCKED
    if account not in self._pending:
        return Result.NO_PENDING
    if not self._accounts[account].codes.verify_and_consume(code):
        self.lockout.record_attempt(account, False, now)
        return Result.BAD_CODE
    self._pending.discard(account)
    self.lockout.record_attempt(account, True, now)
    return Result.OK


def remaining_codes(self, account: str) -> int:
    return self._accounts[account].codes.remaining()
```